ShelfSync

Digital Library Management System

Project Title	ShelfSync Digital Library Management System
Project Type	FullStack Web Application
Technology Stack	Laravel 11, PHP 8.x, MySQL, Blade, Vanilla CSS, Groq AI (Llama 3.1), Google Books API
Submitted By	Mahadi Hassan
Student ID(s)	23235040
Department	Software Engineering
Institution	Daffodil International University
Course	Software Engineering Web Application
Supervisor	Dipta Dipayan Kar
Submission Date	22/04/2026

Table of Contents

1. Executive Summary 3
2. Problem Statement 3
3. Motivation & Rationale 4
4. Proposed Solution Overview 4
5. System Features & Capabilities 5
   5.1 Public Features (No Login Required) 5
   5.2 Student / User Features 5
   5.3 Admin / Librarian Features 6
6. System Architecture 7
7. MVC Component Breakdown 8
   7.1 Controllers 8
   7.2 Models & Eloquent Relationships 8
   7.3 Views 9
   7.4 Route Summary 9
8. Database Design 10
   8.1 Table Schema 10
   8.2 Entity Relationships 11
9. API Architecture 12
   9.1 Groq AI Integration (Llama 3.1 8B Instant) 13
   9.2 DualLayer ISBN Metadata Engine 13
   9.3 Local Cover Engine 14
10. Technology Stack 14
11. Design Patterns & Security 15
   11.1 Design Patterns Applied 15
   11.2 Security Implementation 15
12. Target Audience & Stakeholders 16
13. Project Objectives 16
14. Expected Outcomes & Benefits 16
15. Project Timeline 17
16. Conclusion 17

1. Executive Summary

ShelfSync is a nextgeneration, fully webbased Digital Library Management System designed to replace the paperdriven, inefficient library workflows that are commonplace in universities and colleges across Bangladesh and similar institutions globally. Built on the Laravel 11 PHP framework with a modern glassmorphic user interface and powered by the **Groq AI platform running Meta's Llama 3.1 8B Instant** model, ShelfSync automates the complete lifecycle of library operations from student registration and digital ID card issuance to book rental management, overdue fine calculation, and intelligent book discovery.

The system additionally integrates a **DualLayer Metadata Engine**, a serverside proxy that queries the Google Books API and OpenLibrary API in sequence enabling administrators to add books instantly by ISBN with autopopulated metadata and cover images. A companion **Local Cover Engine** intercepts remote image URLs at save time and downloads them to the server's local filesystem, ensuring book covers load instantly and remain permanently available regardless of external API uptime or network conditions.

The system serves three distinct user groups simultaneously: students who need a fast, mobilefriendly way to browse and rent books; librarians and administrators who need powerful management tools; and unauthenticated guests who can still explore the catalog and interact with the AI book assistant without creating an account. ShelfSync is not merely a CRUD application it is a productiongrade software system with realtime availability tracking, a rulebased AI assistant, an automated fine dispute resolution workflow, a duallayer book metadata engine, and a premium design language that reflects the standards expected of modern web software.

2. Problem Statement

Library management in most educational institutions is still handled through a combination of physical ledgers, handwritten forms, and outdated desktop software. This approach introduces a range of operational, administrative, and user experience problems:

2.1 Physical Library Cards

Students are issued physical library cards that are frequently lost, damaged, or forgotten. Replacing a card is a manual process involving paperwork, photographs, and administrative approval. There is no instant verification mechanism a librarian cannot digitally confirm a student's identity or membership status without physically inspecting the card.

2.2 Manual Book Tracking

Inventory management relies on manual entries in registers or spreadsheets. When a book is borrowed, a librarian manually records it. When it is returned, another entry is made. This process is prone to human error, duplication, and loss of data. There is no realtime visibility into how many copies of a book are available at any given moment.

2.3 No Accountability for Fines

Overdue fine calculation is inconsistent. Librarians must manually check return dates, calculate overdue periods, and inform students. Students often dispute fines without a formal mechanism, and there is no audit trail to resolve disagreements.

2.4 No Digital Discovery Tool

Students cannot search the catalog remotely, check availability before visiting the library, or get information about a book's contents before deciding to rent it. This results in wasted trips and underutilization of the library's collections.

2.5 Administrative Bottlenecks

Administrators have no consolidated dashboard. Approving rental requests, managing returns, handling appeals, and generating reports all require separate tools or manual processes, leading to delays and frustration for both staff and students.

2.6 Absence of Intelligent Assistance

No existing library system used in typical university settings provides AIpowered book summaries or Q&A, leaving students without a guided discovery experience.

2.7 Slow and Unreliable Book Data Entry

Adding new books to the system requires manual entry of every field — title, author, category, description, and cover image. There is no ISBNbased autofill capability, making bulk data entry slow and errorprone.

2.8 Broken and Unreliable Book Cover Images

Systems that hotlink to external image APIs suffer from ratelimiting, IP blocking, and broken images over time — especially on campus WiFi networks with restricted outbound traffic.

3. Motivation & Rationale

The motivation for ShelfSync arises directly from the observation of these pain points in real academic environments. A digitized, automated, and aesthetically refined library system can:

- Save hours of administrative time per week
- Eliminate fine disputes through transparent, automated tracking
- Give students 24/7 access to library services from any device
- Reduce paper waste and physical storage overhead
- Create a measurable audit trail for all transactions
- Provide AIguided discovery that encourages more students to read
- Enable instant book data entry via ISBN lookup with permanent local image caching
- Remain reliable on restricted campus networks by routing all API calls through the server

ShelfSync was also motivated by the gap in the local software landscape existing library management systems (such as Koha or OpenBiblio) are complex, desktoporiented, and require significant technical expertise to deploy and maintain. ShelfSync is a clean, cloudready, Laravelbased system that can be deployed on shared hosting with minimal configuration.

4. Proposed Solution Overview

ShelfSync proposes a unified web application that handles every aspect of library management through a browser. The application is structured around the **ModelViewController (MVC)** architectural pattern, ensuring clear separation of concerns and longterm maintainability.

At its core, ShelfSync provides:

- A **publicfacing landing page** with live search, book catalog, and AI assistant access — accessible to anyone without login
- A **student portal** where authenticated users can rent books, manage their digital library card, track reading progress, manage wishlists, and appeal fines
- An **admin panel** where librarians can manage inventory, approve or reject all student actions, handle fines, and view systemwide activity
- A **Groq AI integration** using the Llama 3.1 8B Instant model that provides intelligent, bookspecific information and conversation to any user, with graceful degradation if the network is unavailable
- A **DualLayer Metadata Engine** that fetches book data by ISBN through a secure serverside proxy, bypassing campus firewall restrictions
- A **Local Cover Engine** that permanently caches book cover images to local storage at save time

All user interactions are secured with CSRF protection, rolebased middleware, and serverside validation. All API keys are stored serverside and never exposed to the browser.

5. System Features & Capabilities

5.1 Public Features (No Login Required)

- **Landing Page & Book Discovery:** A visually rich homepage showcasing featured books, platform features, a howitworks timeline, and a FAQ section with live system metrics.
- **Live Book Search:** A realtime search bar that queries the database as the user types, returning matching results by title, author, and category.
- **Book Catalog Browsing:** A filterable grid of all books with locally cached cover images, category, author, price, and realtime availability count.
- **AI Book Assistant:** By clicking an info icon (ⓘ) on any book card, any user (including guests) can open a modal powered by Llama 3.1 that generates structured metadata and enables a restricted Q&A chat about the specific book.
- **Contact Form:** A public form for submitting inquiries, stored in the database for admin review.

5.2 Student / User Features

- **Registration & Login:** Streamlined authentication flow via Laravel Breeze, optimized for local deployment without email verification overhead.
- **Profile Management:** Students can update their name, email, address, and password.
- **Book Rental Submission:** Authenticated students with an approved library card can request to rent any available book by specifying a return date.
- **My Rentals Dashboard:** A personal view of all active, past, approved, and rejected rentals with fine information and appeal options.
- **Digital Library Card Application:** Students apply for a digital library card; upon admin approval, a glassmorphic digital identity card is generated with 6month validity and autoexpiry.
- **Card Renewal:** Students can request renewal of an expired or revoked card.

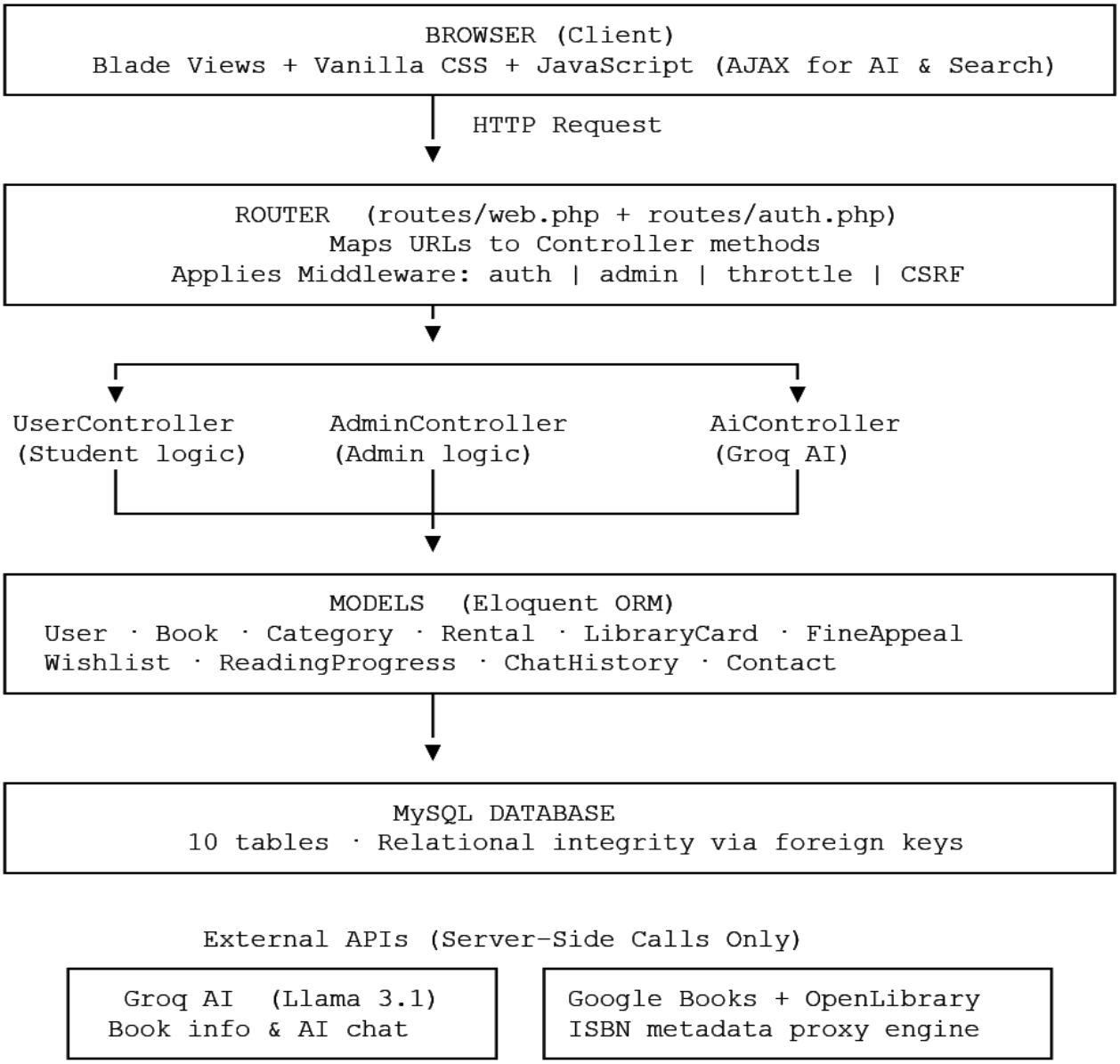
- **Reading Progress Tracker:** Students can log books they are actively reading, update their progress percentage, and mark books as completed.
- **Wishlist:** An AJAXpowered toggle that allows students to bookmark books for future renting without page reload.
- **Persistent AI Chat:** AI conversations are saved perbook in the database for loggedin users, allowing them to resume previous discussions.
- **Fine Appeal Submission:** Students can submit a written appeal for any disputed fine, which enters an admin review queue.

5.3 Admin / Librarian Features

- **Admin Dashboard:** Consolidated overview of total users, books, active rentals, pending requests, overdue count, and recent activity feed.
- **Rental Approval Workflow:** Admins approve or reject rental requests; marking a rental as returned automatically restocks the book inventory.
- **DualLayer Book Fetcher:** Admins can add books by entering an ISBN, the system queries Google Books API first, falls back to OpenLibrary if needed, and autofills all fields including title, author, category, description, and cover image.
- **Local Cover Engine:** When a book is saved, the system automatically downloads the remote cover image and stores it locally in `public/img/covers/`, ensuring covers never break due to external API changes.
- **Returns Scanner:** Dedicated interface for processing book returns by rental ID.
- **Book Management (CRUD):** Full create, read, update, delete for books with metadata and local cover storage.
- **Category Management:** Add or delete book categories used for filtering and classification.
- **Library Card Management:** Approve, reject, or revoke digital library cards with configurable expiry dates.
- **Fine Management:** View, clear, or manually adjust outstanding fines per rental record.
- **Fine Appeal Resolution:** Review and resolve or reject student fine disputes with full audit trail.
- **Member Management:** View all users, promote to admin role, or demote from admin.

6. System Architecture

ShelfSync follows the **MVC (ModelViewController)** pattern as enforced by the Laravel framework.



Key Architectural Decisions:

- All API keys are stored in `.env` and never sent to the browser.
- All AI endpoints are ratelimited to 15 requests per minute per IP.
- The ISBN metadata proxy routes requests through Laravel, bypassing campus firewall CORS restrictions.
- The Local Cover Engine downloads remote images to local disk at save time, eliminating hotlink dependency.
- Blade components are reused across user and admin layouts for consistency.
- Rolebased access control is implemented through a custom `AdminMiddleware` checking the `role` field on the authenticated user.

- Graceful degradation is built into the AI layer, if Groq is unreachable, the system returns real database data instead of showing an error.

7. MVC Component Breakdown

7.1 Controllers

Controller	Methods	Responsibility
`UserController`	13	Landing page, book catalog, rental, reading tracker, wishlist, library card, fine appeals, contact
`AdminController`	20+	Dashboard, rental approval, book CRUD, ISBN proxy, category CRUD, card management, fines, members
`AiController`	4	AI book info card, AI multiturn chat, load chat history, clear chat history
`ProfileController`	3	Show profile form, save profile updates, delete account
`RegisteredUserController`	2	Show registration form, process and store new user
`AuthenticatedSessionController`	3	Show login form, authenticate user, destroy session
`PasswordController`	1	Update user password

7.2 Models & Eloquent Relationships

Model	Table	Key Relationships
`User`	`users`	hasOne → LibraryCard, hasMany → Rental
`Book`	`books`	belongsTo → Category, hasMany → Rental
`Category`	`categories`	hasMany → Book
`Rental`	`rentals`	belongsTo → User, belongsTo → Book, hasOne → FineAppeal
`LibraryCard`	`library_cards`	belongsTo → User
`FineAppeal`	`fine_appeals`	belongsTo → Rental, belongsTo → User
`ChatHistory`	`chat_histories`	belongsTo → User, belongsTo → Book
`ReadingProgress`	`reading_progress`	belongsTo → User, belongsTo → Book
`Wishlist`	`wishlists`	belongsTo → User, belongsTo → Book
`Contact`	`contacts`	belongsTo → User

7.3 Views

Directory	Views	Purpose
`views/`	1	`welcome.blade.php` — Landing page
`views/auth/`	2	`login.blade.php`, `register.blade.php`
`views/user/`	6	Studentfacing pages (catalog, dashboard, rent, myrents, contact, index)
`views/admin/`	10	Admin pages (dashboard, books, rentals, cards, fines, categories, members, scanner)
`views/layouts/`	5	Master layout templates (user, admin, app, guest, navigation)
`views/components/`	11	Reusable UI blocks (AI modal, global effects, form elements)
`views/profile/`	1 + 3 partials	Profile settings page

7.4 Route Summary

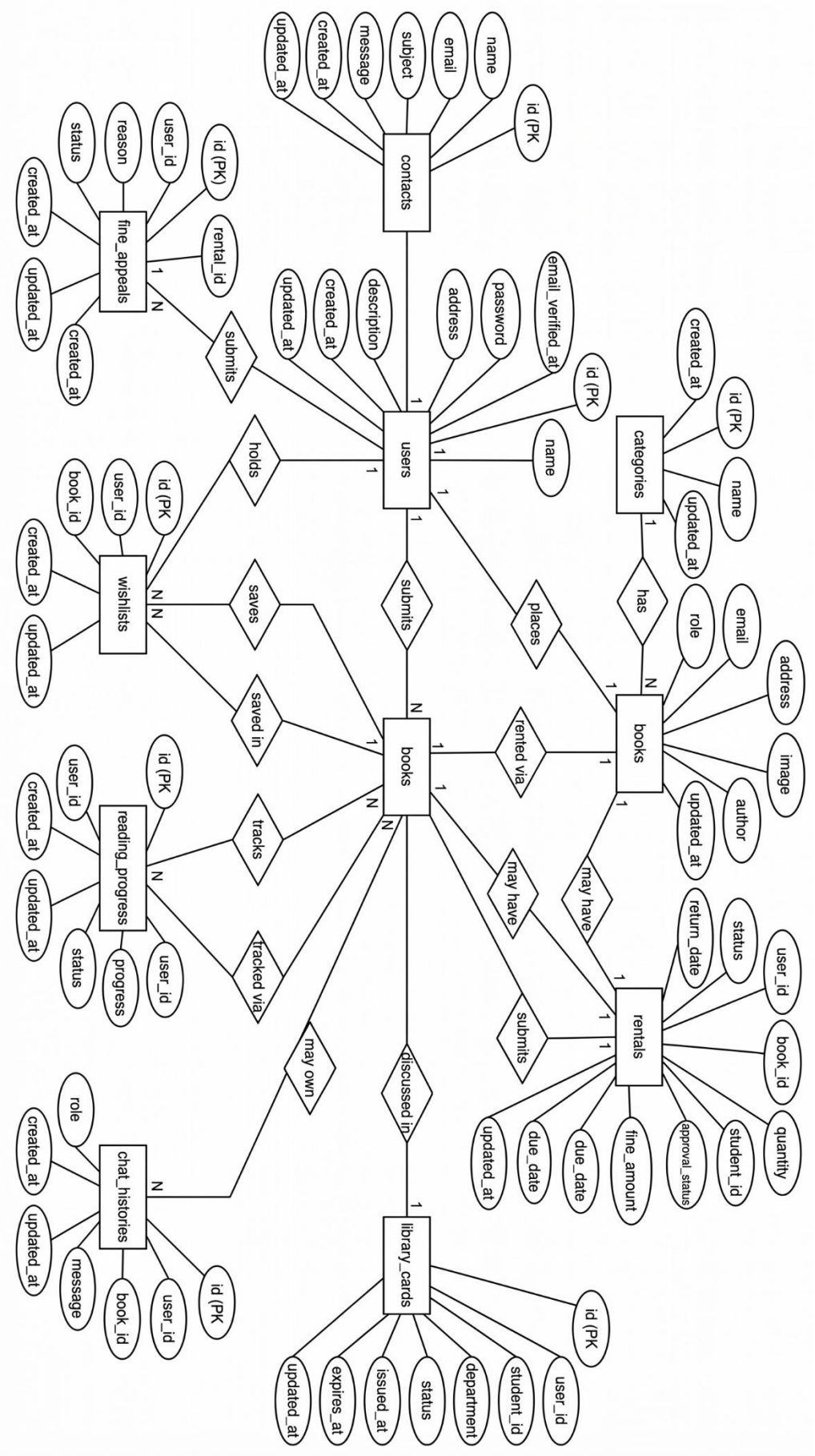
Route Group	Route Count	Middleware Applied	URL Prefix
Public routes	4	None	`/`
AI API routes	4	`throttle:15,1`	`/api/bookai/`
Auth routes	6	`guest` / `auth`	`/`
Student routes	12	`auth`	`/`
Admin routes	20	`auth` + `admin`	`/admin/`
Total	**46**		

8. Database Design

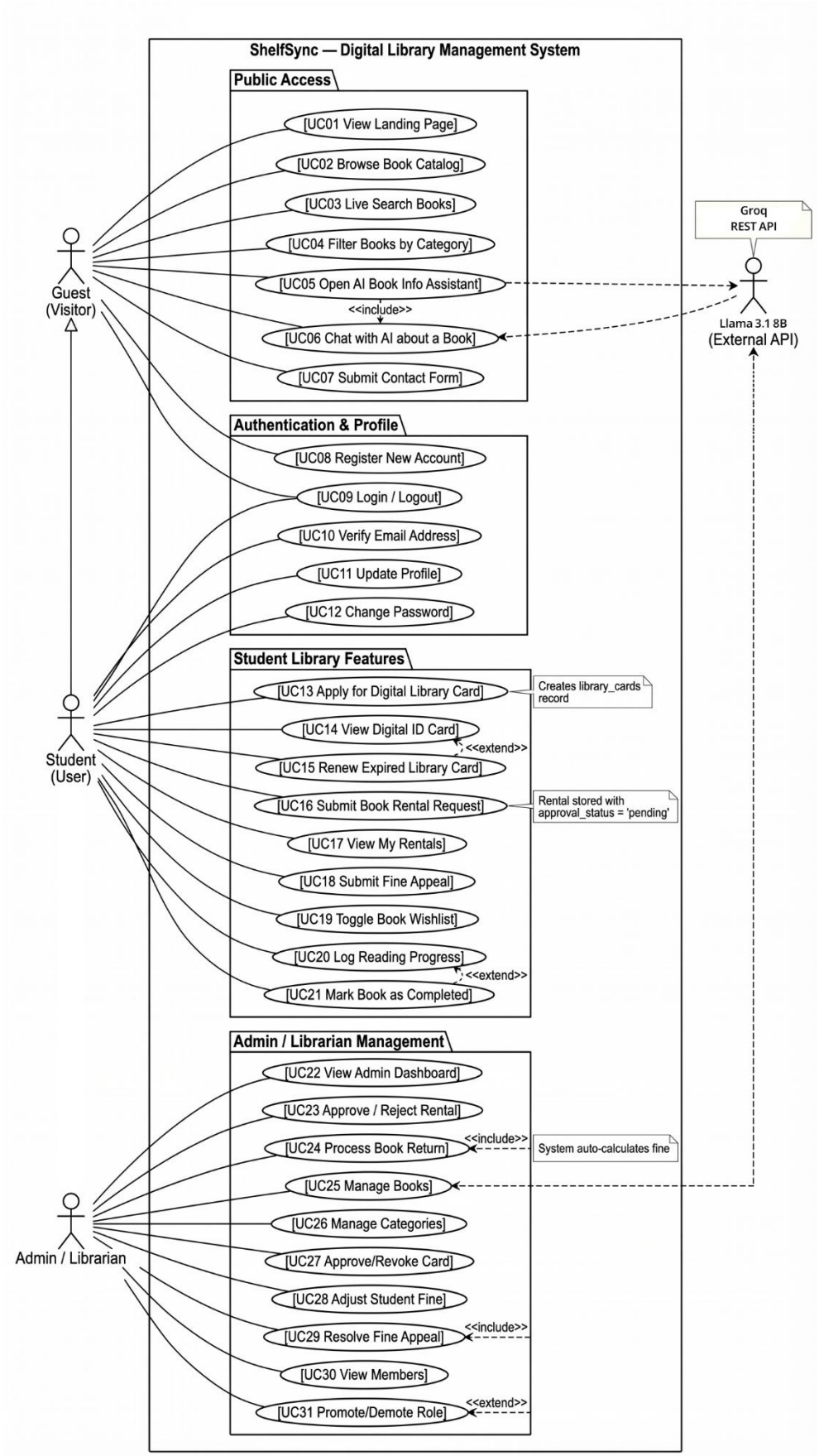
8.1 Table Schema

Table	Key Columns	Foreign Keys
`users`	id, name, email, password, role, address	—
`books`	id, name, author, image, category_id, quantity, price, description	category_id → categories
`categories`	id, name	—
`rentals`	id, user_id, book_id, student_id, quantity, return_date, approval_status, fine_amount, due_date	user_id → users, book_id → books
`library_cards`	id, user_id, student_id, department, status, issued_at, expires_at	user_id → users
`fine_appeals`	id, rental_id, user_id, reason, status	rental_id → rentals, user_id → users
`chat_histories`	id, user_id, book_id, role, message	user_id → users, book_id → books
`reading_progress`	id, user_id, book_id, progress, status, started_at, completed_at	user_id → users, book_id → books
`wishlists`	id, user_id, book_id	user_id → users, book_id → books
`contacts`	id, user_id, subject, phone, email, category, message	user_id → users

8.2 Entity Relationships



8.3 Use Case Diagram



9. API Architecture

9.1 Groq AI Integration (Llama 3.1 8B Instant)

Property	Value
API Endpoint	`https://api.groq.com/openai/v1/chat/completions`
Model	`llama3.18binstant`
Authentication	Bearer token via `GROQ_API_KEY` in `.env`
Request Timeout	10 seconds
Temperature	0.7
Max Tokens	1024 per response
Rate Limiting	15 requests per minute per IP (Laravel throttle middleware)
Graceful Fallback	Returns real database data when API is unavailable — UI never crashes

9.2 DualLayer ISBN Metadata Engine

When an admin enters an ISBN, the system queries two APIs in sequence as a serverside proxy:

Tier	API	Query Endpoint	Timeout
Tier 1 — Primary	Google Books API	`googleapis.com/books/v1/volumes?q=isbn:{isbn}`	5 seconds
Tier 2 — Fallback	OpenLibrary API	`openlibrary.org/api/books?bibkeys=ISBN:{isbn}`	5 seconds

Flow:

```
Admin enters ISBN
        │
        ▼
Laravel /admin/books/fetch-isbn (proxy endpoint)
        │
        ├──► Query Google Books API
        │          Found? ──► Return title, author, category, description, cover
        │          Not found or timeout?
        │
        └──► Query OpenLibrary API
                   Found? ──► Return available fields
                   Not found? ──► Return 404 to browser

Browser auto-fills the book form with returned data.
```

This proxy architecture means the browser never directly contacts external APIs bypassing CORS errors and campus WiFi firewall restrictions that would otherwise block these requests.

9.3 Local Cover Engine

```
Admin saves a book with a remote cover URL
        │
        ▼
storeBook() / updateBook() intercepts the URL
        │
        ├── Is URL a remote http/https link?
        │       No  ──► Store URL as-is in database
        │       Yes ──► Download image via file_get_contents()
        │                      │
        │               Success (size > 500 bytes)?
        │                    Yes ──► Save to public/img/covers/cover_{uid}.jpg
        │                    No  ──► Store original URL as fallback
        │
        └──► Database stores local asset() URL
             Result: Cover loads from local disk — immune to external API
downtime
```

10. Technology Stack

Layer	Technology	Version	Justification
Backend Framework	Laravel	11.x	Mature MVC framework with builtin auth, ORM, middleware, and routing
Language	PHP	8.x	Required by Laravel 11; supports modern type declarations
Database	MySQL (via XAMPP)	8.0	Reliable relational database; wellsupported by Eloquent ORM
ORM	Eloquent	Builtin	`hasMany`, `belongsTo` relationships cleanly map to the schema
Authentication	Laravel Breeze	2.x	Lightweight scaffolding for registration and login
Frontend	Blade Templating + Vanilla CSS	Builtin	Custom `ss` design token system; no JS framework overhead
AI Integration	Groq REST API	v1	Ultrafast inference (~200ms), free tier, OpenAIcompatible format
AI Model	Meta Llama 3.1 8B Instant	—	Structured JSON output, multiturn conversation, instruction enforcement
Book Metadata	Google Books + OpenLibrary	REST	Duallayer serverside proxy for maximum ISBN coverage
UI Design Language	Glassmorphism, Dark Mode	CSS	`backdropfilter: blur()`, gradient overlays, CSS animations
Web Server	Apache (XAMPP)	2.4	Standard local PHP hosting environment

11. Design Patterns & Security

11.1 Design Patterns Applied

Pattern	Where Applied in ShelfSync
MVC	Entire application — Models, Views, Controllers strictly separated
Middleware Pipeline	`auth` → `admin` chain on admin routes; `throttle` on AI routes
Proxy Pattern	`AdminController@fetchIsbn` acts as serverside proxy to external APIs
Strategy Pattern	Duallayer engine: Google Books strategy → OpenLibrary fallback strategy
Graceful Degradation	AiController returns database data when Groq API is unavailable
Component Composition	Blade components (`<xbookaimodal>`, `<xglobalfx>`) composed into layouts
AutoExpiry Accessor	`User::getCardStatusAttribute()` autoexpires library cards at readtime
AJAX Toggle	Wishlist uses async JavaScript for instant toggling without page reload

11.2 Security Implementation

Security Measure	Implementation
CSRF Protection	All forms include `@csrf`; AJAX calls include `XCSRFTOKEN` header
RoleBased Access Control	Admin routes doubleprotected by `auth` + custom `AdminMiddleware`
ServerSide API Keys	Groq key in `.env`, accessed via `config()` - never sent to browser
Backend Proxy	ISBN metadata fetched serverside to bypass CORS and firewall blocks
Password Hashing	Bcrypt via Laravel's `Hash::make()` - plaintext never stored
Input Validation	All inputs validated via `$request>validate()` before any DB write
Rate Limiting	AI endpoints throttled at 15 req/min per IP via Laravel middleware
SQL Injection Prevention	Eloquent ORM parameterizes all queries; no raw SQL in the codebase
Mass Assignment Protection	All models define explicit `$fillable` arrays
Ownership Verification	Student queries always filtered by `Auth::id()` to prevent crossuser access

12. Target Audience & Stakeholders

Stakeholder	Role in the System	Key Needs
Students	Primary users who rent books and manage their library membership	Easy browsing, fast rental, transparent fines, mobile access
Librarians / Admins	Operators managing inventory and approving requests	Efficient workflows, consolidated dashboard, audit trail
University Institution	Owner of the system	Reduced paper costs, accountability, scalability
Guests / Visitors	Prospective students or general public	Catalog access and AI book discovery without registration

13. Project Objectives

1. Digitize the complete library membership lifecycle (registration → card issuance → expiry → renewal → revocation).
2. Automate book rental approval, return processing, and fine calculation with realtime inventory adjustment.
3. Eliminate physical library cards with a databasebacked digital identity card featuring autoexpiry.
4. Enable realtime book availability so students always see accurate stock counts.
5. Integrate an AIpowered book assistant (Llama 3.1) accessible to all users including guests.
6. Provide a transparent fine management and dispute resolution system with a full audit trail.
7. Implement a duallayer ISBN metadata engine for instant, autofilled book data entry.
8. Cache all book cover images locally to ensure permanent, fastloading availability on any network.
9. Deliver a premium, responsive glassmorphic user interface reflecting professional web standards.
10. Enforce security best practices: CSRF protection, rolebased access control, serverside API key management, and rate limiting.

14. Expected Outcomes & Benefits

Benefit	Details
Time Savings	Admin approval workflows reduce pertransaction time from 10+ minutes (manual) to under 30 seconds.
Accuracy	Automated fine calculation eliminates arithmetic errors; inventory is always realtime accurate.
Accessibility	24/7 access from any device — no need to physically visit the library to check availability.
Transparency	Students can see their fine history, appeal status, and card status in real time.
AI Discovery	Students are guided to explore new books through an AI assistant that explains themes and answers questions.

Benefit	Details
Instant Book Entry	ISBN lookup populates all book metadata in under 2 seconds via the duallayer engine.
Reliable Images	Local cover engine ensures book covers never break due to external API downtime or network blocking.
Paperless Operation	All forms, cards, and records are digital, reducing physical storage and paper costs.
Network Resilience	Backend proxy architecture works reliably even on throttled or restricted campus WiFi networks.
Scalability	The system can be extended to support multiple branches, barcode scanners, or email notifications with minimal changes.

15. Project Timeline

Phase	Activities	Duration
Phase 1 — Planning	Requirements gathering, database design, wireframing	Week 1–2
Phase 2 — Database & Models	Migrations, Eloquent models, relationships	Week 3
Phase 3 — Auth & Roles	Breeze setup, AdminMiddleware, role management	Week 4
Phase 4 — Core Features	Rental system, fines, library card, wishlist, reading progress tracker	Week 5–7
Phase 5 — Admin Panel	Dashboard, all CRUD operations, approval workflows, member management	Week 8–9
Phase 6 — AI Integration	Groq API, AI modal UI, multiturn chat, graceful degradation	Week 10
Phase 7 — Metadata Engine	Duallayer ISBN proxy, local cover engine, network optimization	Week 11
Phase 8 — UI Polish & Testing	Glassmorphism design system, animations, edgecase testing, cleanup	Week 12

16. Conclusion

ShelfSync represents a significant step forward from the manual, paperbased library systems currently in use at many academic institutions. By combining a robust Laravel backend, a thoughtfully designed relational database with 10 interconnected tables, the intelligence of Groq's Llama 3.1 AI, and a duallayer metadata engine backed by Google Books and OpenLibrary, the project delivers a solution that is not only functionally complete but also architecturally resilient and aesthetically modern.

Every feature from the glassmorphic digital library card to the AIpowered book assistant to the local cover caching engine was designed with real user needs in mind. The system employs established software design patterns (MVC, Proxy Pattern, Strategy Pattern, Graceful Degradation) and enforces comprehensive security measures across all layers.